Indian Institute of Technology Gandhinagar

Department of Computer Science and Engineering

Lexical Analyzer for a C-Like Language Using Lex/Flex

Course: Compiler Design
Instructor: Shouvick Mondal
Group ID: 22

February 19, 2026

Contents

1 Introduction **2**

2 Token Design and Data Structures **2**

3 Regular Definitions and Pattern Design **2**

4 Token Recognition Approach **3**

4.1 Keywords 3

4.2 Identifiers 3

4.3 Literals 3

4.4 Operators and Delimiters 4

5 Comments and Whitespace Handling **4**

6 Lex Start Conditions and State Management **5**

7 Conflict Resolution Strategy **5**

8 Error Handling Mechanism **5**

9 Position Tracking **6**

10 Output Format **6**

11 Implementation and Build Process **6**

12 Scope and Limitations **6**

12.1 Scope 6

12.2 Limitations 7

13 Conclusion **7**

1 Introduction

In this project, we implemented a lexical analyzer for a simplified subset of the C programming language using Lex/Flex. The lexical analyzer is responsible for reading the source code as a stream of characters and converting it into meaningful tokens that can be processed by later compilation stages.

We began by defining a language specification describing all supported tokens such as keywords, identifiers, literals, operators, delimiters, and comments. Based on this specification, we designed regular expressions for each token category and implemented them using Flex rules. Flex automatically generated a deterministic finite automaton (DFA) that scans the input efficiently.

The primary objective of this project was to gain practical experience in lexical analysis, understand the relationship between regular expressions and automata, and implement a working scanner capable of error detection and structured token output.

2 Token Design and Data Structures

During implementation, we designed a token abstraction that stores token type, lexeme, line number, and column number,length. This approach enabled structured token generation and precise error reporting. The token categories implemented include keywords, identifiers, literals, operators, delimiters, comments, whitespace, and unknown tokens.

```
void print_token(const char* token_type, const char* lexeme, int line, int column, int length) {
    printf("%-20s %-15s %4d %4d %4d\n", token_type, lexeme, line, column, length);
}
```

3 Regular Definitions and Pattern Design

We defined reusable regular expressions to simplify lexer rules and improve readability. These definitions include patterns for letters, digits, identifiers, integers, and floating constants. Using regular definitions helped reduce duplication and made the lexer easier to maintain. Flex internally converts these regular expressions into deterministic finite automata, ensuring linear-time scanning.

```
/* Definitions */
DIGIT           [0-9]
LETTER          [A-Za-z]
IDENTIFIER      [A-Za-z_][A-Za-z0-9_]*
DECIMAL_INT     [0-9]+
HEX_INT         0[xX][0-9A-Fa-f]+
FLOAT_CONST     [0-9]*\.[0-9]+
WHITESPACE      [ \t\r]+
NEWLINE         \n
```

4 Token Recognition Approach

4.1 Keywords

We implemented keyword recognition by explicitly defining patterns for reserved words such as int, long, short, float, double, void, char, struct, extern, if, else, switch, case, default, for, while, do, break, continue, return, and sizeof.

```
/* ===== KEYWORDS ===== */
"int"               { print_token("INT", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"long"              { print_token("LONG", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"short"             { print_token("SHORT", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"float"             { print_token("FLOAT", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"double"            { print_token("DOUBLE", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"void"              { print_token("VOID", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"char"              { print_token("CHAR", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"struct"            { print_token("STRUCT", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"extern"            { print_token("EXTERN", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"if"                { print_token("IF", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"else"              { print_token("ELSE", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"switch"            { print_token("SWITCH", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"case"              { print_token("CASE", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"default"           { print_token("DEFAULT", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"for"               { print_token("FOR", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"while"             { print_token("WHILE", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"do"                { print_token("DO", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"break"             { print_token("BREAK", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"continue"          { print_token("CONTINUE", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"return"            { print_token("RETURN", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"sizeof"            { print_token("SIZEOF", yytext, line_number, column_number, yyleng); column_number += yyleng; }
```

4.2 Identifiers

Identifiers were designed to start with a letter or underscore and may contain letters, digits, and underscores, ensuring accurate classification of user-defined names.

```
IDENTIFIER      [A-Za-z_][A-Za-z0-9_]*
```

4.3 Literals

Support was implemented for integer constants, floating constants, character constants, and string literals with escape sequence handling.

```
    /* ===== CONSTANTS ===== */
    /* Hexadecimal Integer (must come before decimal) */
{HEX_INT}               { print_token("HEX_CONSTANT", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Floating Point Constant */
{FLOAT_CONST}           { print_token("FLOAT_CONSTANT", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Decimal Integer */
{DECIMAL_INT}           { print_token("INTEGER_CONSTANT", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Character Constant */
'([^'\\]|\\n|\\t|\\r|\\0|\\\\|\\')'  { print_token("CHAR_CONSTANT", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* String Literal */
\"([^\"\\\n]|\\.)*\"    { print_token("STRING_LITERAL", yytext, line_number, column_number, yyleng); column_number += yyleng; }
```

4.4 Operators and Delimiters

Arithmetic, relational, logical, assignment, and bitwise operators were implemented along with delimiters such as parentheses, braces, brackets, commas, and semicolons.

```
    /* ===== OPERATORS ===== */
    /* Multi-character operators (must come before single-character) */

    /* Special Operator */
"<=>"                   { print_token("SPACESHIP", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Structure Access Operators */
"->"                    { print_token("ARROW", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Increment/Decrement Operators */
"++"                    { print_token("INCREMENT", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"--"                    { print_token("DECREMENT", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Relational Operators - Two character */
"<="                    { print_token("LESS_EQUAL", yytext, line_number, column_number, yyleng); column_number += yyleng; }
">="                    { print_token("GREATER_EQUAL", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"=="                    { print_token("EQUAL", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"!="                    { print_token("NOT_EQUAL", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Logical Operators */
"&&"                    { print_token("LOGICAL_AND", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"||"                    { print_token("LOGICAL_OR", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Arithmetic Operators - Single character */
"+"                     { print_token("PLUS", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"-"                     { print_token("MINUS", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"*"                     { print_token("MULTIPLY", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"/"                     { print_token("DIVIDE", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"%"                     { print_token("MODULO", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Relational Operators - Single character */
"<"                     { print_token("LESS_THAN", yytext, line_number, column_number, yyleng); column_number += yyleng; }
">"                     { print_token("GREATER_THAN", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Bitwise Operators */
"&"                     { print_token("BITWISE_AND", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"|"                     { print_token("BITWISE_OR", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"^"                     { print_token("BITWISE_XOR", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"~"                     { print_token("BITWISE_NOT", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Assignment Operator */
"="                     { print_token("ASSIGN", yytext, line_number, column_number, yyleng); column_number += yyleng; }

    /* Structure Access - Dot */
"."                     { print_token("DOT", yytext, line_number, column_number, yyleng); column_number += yyleng; }


    /* ===== DELIMITERS / PUNCTUATORS ===== */
"("                     { print_token("LPAREN", yytext, line_number, column_number, yyleng); column_number += yyleng; }
")"                     { print_token("RPAREN", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"{"                     { print_token("LBRACE", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"}"                     { print_token("RBRACE", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"["                     { print_token("LBRACKET", yytext, line_number, column_number, yyleng); column_number += yyleng; }
"]"                     { print_token("RBRACKET", yytext, line_number, column_number, yyleng); column_number += yyleng; }
";"                     { print_token("SEMICOLON", yytext, line_number, column_number, yyleng); column_number += yyleng; }
","                     { print_token("COMMA", yytext, line_number, column_number, yyleng); column_number += yyleng; }
":"                     { print_token("COLON", yytext, line_number, column_number, yyleng); column_number += yyleng; }
```

5 Comments and Whitespace Handling

We implemented both single-line and multi-line comments. Multi-line comments were handled using start conditions to ensure correct detection of comment boundaries. Unclosed comments generate lexical errors. Whitespace characters were ignored while maintaining line and column tracking.

```
    /* ===== COMMENTS ===== */
    /* Single-line comment */
"//".*                  { column_number += yyleng; /* Ignore single-line comments */ }

    /* Multi-line comment start */
"/*"                    {
                          comment_start_line = line_number;
                          column_number += 2;
                          BEGIN(COMMENT);
                        }

<COMMENT>"*/"           { column_number += 2; BEGIN(INITIAL); }
<COMMENT>\n             { line_number++; column_number = 1; }
<COMMENT>.              { column_number++; /* Ignore comment content */ }
<COMMENT><<EOF>>        {
                          fprintf(stderr, "*** lexing terminated *** [lexer error]: ill-formed comment starting at line %d\n", comment_start_line);
                          return 0;
                        }
```

6 Lex Start Conditions and State Management

Lex start conditions were used to manage contexts such as multi-line comments and string literals, ensuring proper token boundary detection and improved error handling.

7 Conflict Resolution Strategy

Conflicts between overlapping token patterns were resolved using longest-match semantics and rule ordering priority provided by Flex. This ensured deterministic token classification.

8 Error Handling Mechanism

Error detection was implemented for unclosed comments, invalid tokens, and ill-formed literals. Errors are reported with line and column information to assist debugging.

```
aniruddh_mamilla@ANIRUDDH:/mnt/c/Users/mamil/Downloads/compilers$ ./lexer test_program3.c
TOKEN_TYPE          LEXEME          LINE  COL  LEN
---------------------------------------------------------------------------
SINGLE_LINE_CMT     // Test Program 3 - Functions and Pointers    1    1   42
EXTERN              extern            3    1    6
VOID                void              3    8    4
IDENTIFIER          external_func     3   13   13
LPAREN              (                 3   26    1
RPAREN              )                 3   27    1
SEMICOLON           ;                 3   28    1
INT                 int               5    1    3
IDENTIFIER          add               5    5    3
LPAREN              (                 5    8    1
INT                 int               5    9    3
IDENTIFIER          a                 5   13    1
COMMA               ,                 5   14    1
INT                 int               5   16    3
IDENTIFIER          b                 5   20    1
RPAREN              )                 5   21    1
LBRACE              {                 5   23    1
RETURN              return            6    5    6
IDENTIFIER          a                 6   12    1
PLUS                +                 6   14    1
IDENTIFIER          b                 6   16    1
SEMICOLON           ;                 6   17    1
RBRACE              }                 7    1    1
*** lexing terminated *** [lexer error]: ill-formed comment starting at line 8
```

9 Position Tracking

Line and column counters were maintained dynamically as the input stream was processed, enabling precise token location reporting and improved diagnostics.

10 Output Format

Tokens are printed as:

```
<TOKEN_TYPE, LEXEME, LINE, COLUMN, LENGTH>
```

```
aniruddh_mamilla@ANIRUDDH:/mnt/c/Users/mamil/Downloads/compilers$ ./lexer test_program.c
TOKEN_TYPE          LEXEME          LINE  COL  LEN
--------------------------------------------------------------------------------
INT                 int               3    1    3
IDENTIFIER          main              3    5    4
LPAREN              (                 3    9    1
RPAREN              )                 3   10    1
LBRACE              {                 3   12    1
INT                 int               4    5    3
IDENTIFIER          x                 4    9    1
ASSIGN              =                 4   11    1
INTEGER_CONSTANT    10                4   13    2
SEMICOLON           ;                 4   15    1
FLOAT               float             5    5    5
IDENTIFIER          y                 5   11    1
ASSIGN              =                 5   13    1
FLOAT_CONSTANT      3.14              5   15    4
SEMICOLON           ;                 5   19    1
CHAR                char              6    5    4
IDENTIFIER          ch                6   10    2
ASSIGN              =                 6   13    1
CHAR_CONSTANT       'a'               6   15    3
SEMICOLON           ;                 6   18    1
```

11 Implementation and Build Process

The lexical analyzer was implemented using Flex for scanner generation, GCC for compilation, and a Makefile for automation.

```
make
./lexer input.c
```

12 Scope and Limitations

12.1 Scope

- Demonstrates practical lexical analysis

- Implements token recognition using regex
- Provides error detection and token output
- Serves as foundation for parser implementation

12.2 Limitations

- Restricted C subset
- No preprocessing directives
- Limited operator support
- No syntax or semantic analysis

13 Conclusion

In this project, we successfully designed and implemented a lexical analyzer for a simplified C-like language using Lex/Flex. The system effectively recognizes tokens, handles comments and whitespace, tracks token positions, and detects lexical errors. The project provided practical experience with regular expressions, finite automata, and scanner generation tools, forming a strong foundation for future compiler development.